Universidad Da Vinci De Guatemala

Facultad de Ingeniería

Carrera: Ingeniería en Sistemas

“Primer Examen Parcial”

Gelber Gerardo González Rendon

Carné: 202500187

Curso: Estructura de Datos

Guatemala, marzo de 2026

Introducción

La notación Big-O, también conocida como “O grande”, constituye una herramienta fundamental dentro de la teoría de algoritmos y la informática. Su propósito es ofrecer un marco matemático que permita analizar y expresar el rendimiento de un procedimiento sin depender de factores externos como la velocidad específica de una computadora o las condiciones del entorno de ejecución. En lugar de enfocarse en medir tiempos exactos en segundos que pueden variar según el hardware o el sistema operativo, la notación Big-O describe cómo se comporta el consumo de recursos (ya sea tiempo de ejecución o memoria utilizada) a medida que crece el tamaño de los datos de entrada, representado habitualmente por la variable n.

De esta manera, Big-O no busca dar una cifra precisa, sino capturar la tendencia de crecimiento del algoritmo: cómo escala su complejidad cuando los problemas se vuelven más grandes. Gracias a esta abstracción, los científicos de la computación y los desarrolladores pueden comparar algoritmos, identificar cuellos de botella y elegir soluciones más eficientes para escenarios donde el volumen de datos es significativo

Algoritmos Desarrollados

A1. Cálculo del Factorial

El factorial de un número entero positivo n se obtiene multiplicando todos los valores desde 1 hasta n.

- **Enfoque Iterativo:** Se emplea un ciclo que acumula el producto en una variable. Es un método sencillo, directo y con buen desempeño.

- **Enfoque Recursivo:** El problema se formula en términos de sí mismo: el factorial de n depende del factorial de n-1. Aunque elegante, puede ser menos eficiente por el costo de las llamadas recursivas.

A2. Serie de Fibonacci

La sucesión de Fibonacci se construye sumando los dos números anteriores para obtener el siguiente. El objetivo es calcular el valor en la posición n.

- **Versión Iterativa:** Utiliza dos variables que van actualizándose en cada paso, lo que permite avanzar rápidamente en la secuencia con complejidad lineal O(n).

- **Versión Recursiva:** Es el ejemplo clásico de cómo la recursión puede volverse costosa. Cada llamada genera dos más, formando un árbol de cálculos repetidos que crece de manera exponencial O(2^n).

A3. Búsqueda Secuencial

Este método consiste en localizar un elemento dentro de una lista recorriéndola de principio a fin.

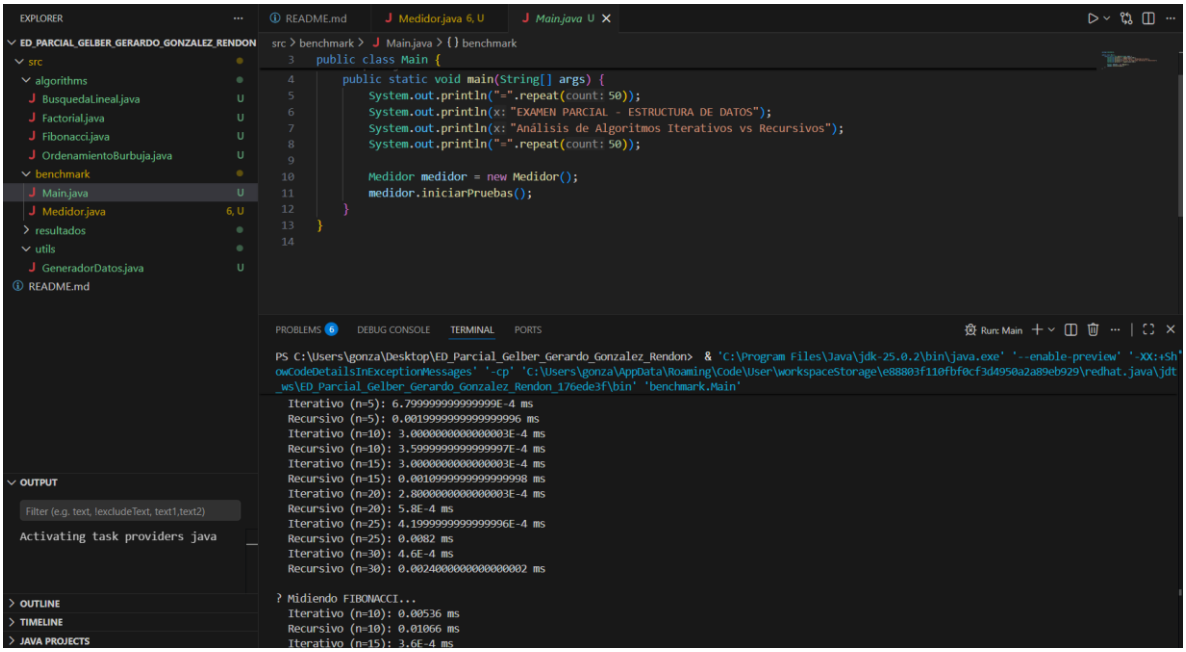
- Funcionamiento: Se examina cada posición y se compara con el valor buscado. Si se encuentra, se devuelve el índice; si no, el resultado es -1. Tanto la versión iterativa como la recursiva mantienen una complejidad lineal O(n).

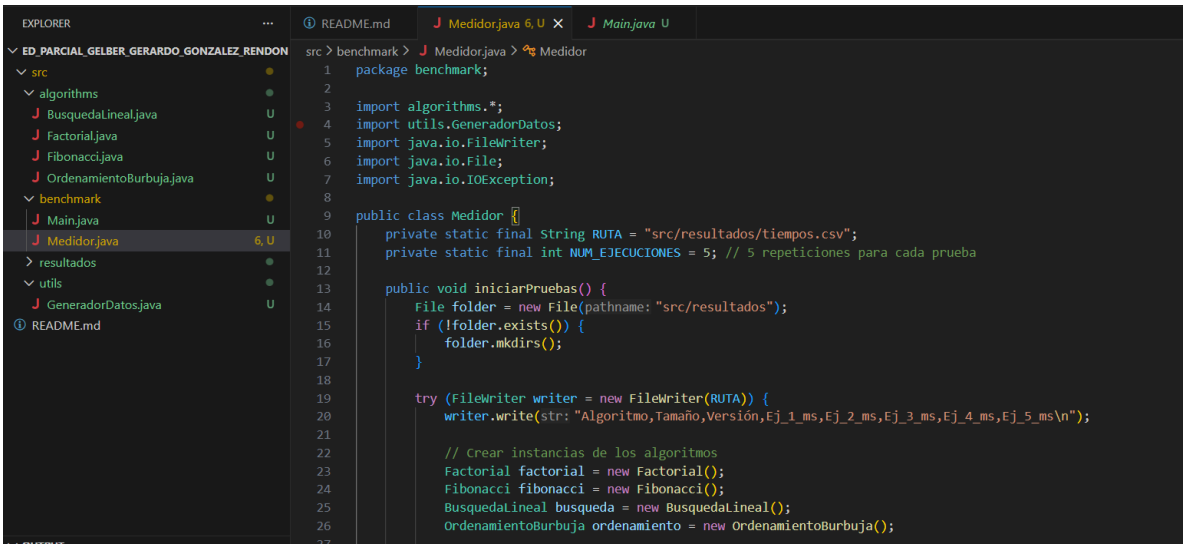
A4. Método de la Burbuja

Un algoritmo de ordenamiento elemental que organiza los datos mediante intercambios sucesivos.

- Funcionamiento: Se comparan pares de elementos contiguos y, si están en el orden incorrecto, se intercambian. Este proceso se repite varias veces, como burbujas ascendiendo en agua, hasta que los valores quedan ordenados. Debido a sus ciclos anidados, su eficiencia es cuadrática O(n^2).

Código Java


EXPLORER
ED_PARCIAL_GELBER_GERARDO_GONZALEZ_RENDON
src
algorithms
BusquedaLineal.java
Factorial.java
Fibonacci.java
OrdenamientoBurbuja.java
benchmark
Main.java
Medidor.java
resultados
utils
GeneradorDatos.java
README.md
OUTPUT
Activating task providers java
OUTLINE
TIMELINE
JAVA PROJECTS
README.md
Medidor.java 6, U
Main.java U
src > benchmark > Main.java > benchmark
public class Main {
public static void main(String[] args) {
System.out.println("=".repeat(count: 50));
System.out.println(x: "EXAMEN PARCIAL - ESTRUCTURA DE DATOS");
System.out.println(x: "Análisis de Algoritmos Iterativos vs Recursivos");
System.out.println("=".repeat(count: 50));
Medidor medidor = new Medidor();
medidor.iniciarPruebas();
}
}
PROBLEMS 6
DEBUG CONSOLE
TERMINAL
PORTS
Run: Main
PS C:\Users\gonza\Desktop\ED_Parcial_Gelber_Gerardo_Gonzalez_Rendon> & 'C:\Program Files\Java\jdk-25.0.2\bin\java.exe' '--enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\gonza\AppData\Roaming\Code\User\workspaceStorage\e88803f110fbf0cf3d4950a2a89eb929\redhat.java\jdt_ws\ED_Parcial_Gelber_Gerardo_Gonzalez_Rendon_176ede3f\bin' 'benchmark.Main'
Iterativo (n=5): 6.799999999999999E-4 ms
Recursivo (n=5): 0.001999999999999996 ms
Iterativo (n=10): 3.0000000000000003E-4 ms
Recursivo (n=10): 3.5999999999999997E-4 ms
Iterativo (n=15): 3.0000000000000003E-4 ms
Recursivo (n=15): 0.0010999999999999998 ms
Iterativo (n=20): 2.8000000000000003E-4 ms
Recursivo (n=20): 5.8E-4 ms
Iterativo (n=25): 4.1999999999999996E-4 ms
Recursivo (n=25): 0.0082 ms
Iterativo (n=30): 4.6E-4 ms
Recursivo (n=30): 0.0024000000000000002 ms
? Midiendo FIBONACCI...
Iterativo (n=10): 0.00536 ms
Recursivo (n=10): 0.01066 ms
Iterativo (n=15): 3.6E-4 ms


EXPLORER
ED_PARCIAL_GELBER_GERARDO_GONZALEZ_RENDON
src
algorithms
BusquedaLineal.java
Factorial.java
Fibonacci.java
OrdenamientoBurbuja.java
benchmark
Main.java
Medidor.java
resultados
utils
GeneradorDatos.java
README.md
README.md
Medidor.java 6, U
Main.java U
src > benchmark > Medidor.java > Medidor
package benchmark;
import algorithms.*;
import utils.GeneradorDatos;
import java.io.FileWriter;
import java.io.File;
import java.io.IOException;
public class Medidor {
private static final String RUTA = "src/resultados/tiempos.csv";
private static final int NUM_EJECUCIONES = 5; // 5 repeticiones para cada prueba
public void iniciarPruebas() {
File folder = new File(pathname: "src/resultados");
if (!folder.exists()) {
folder.mkdirs();
}
try (FileWriter writer = new FileWriter(RUTA)) {
writer.write(str: "Algoritmo,Tamaño,Versión,Ej_1_ms,Ej_2_ms,Ej_3_ms,Ej_4_ms,Ej_5_ms\n");
// Crear instancias de los algoritmos
Factorial factorial = new Factorial();
Fibonacci fibonacci = new Fibonacci();
BusquedaLineal busqueda = new BusquedaLineal();
OrdenamientoBurbuja ordenamiento = new OrdenamientoBurbuja();

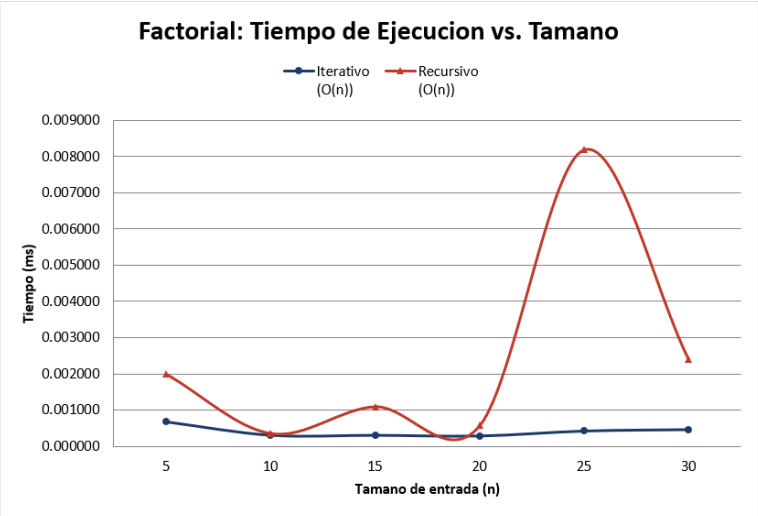
HOJA 1 — Datos Crudos | Tiempos de Ejecucion (ms)								
Algoritmo	**Version**	**n**	**Ej. 1 (ms)**	**Ej. 2 (ms)**	**Ej. 3 (ms)**	**Ej. 4 (ms)**	**Ej. 5 (ms)**	**PROMEDIO (ms)**
A1 - Factorial	Iterativo	5	0.003	0.0002	0.0001	0.0001	0	**0.00068**
A1 - Factorial	Iterativo	10	0.001	0.0002	0.0001	0.0001	0.0001	**0.0003**
A1 - Factorial	Iterativo	15	0.0007	0.0002	0.0002	0.0002	0.0002	**0.0003**
A1 - Factorial	Iterativo	20	0.0007	0.0002	0.0002	0.0002	0.0001	**0.00028**
A1 - Factorial	Iterativo	25	0.0007	0.0004	0.0004	0.0003	0.0003	**0.00042**
A1 - Factorial	Iterativo	30	0.0009	0.0004	0.0004	0.0003	0.0003	**0.00046**
A1 - Factorial	Recursivo	5	0.0095	0.0002	0.0001	0.0001	0.0001	**0.002**
A1 - Factorial	Recursivo	10	0.0007	0.0003	0.0003	0.0003	0.0002	**0.00036**
A1 - Factorial	Recursivo	15	0.0011	0.0006	0.0005	0.0028	0.0005	**0.0011**
A1 - Factorial	Recursivo	20	0.0009	0.0005	0.0006	0.0005	0.0004	**0.00058**
A1 - Factorial	Recursivo	25	0.0381	0.0007	0.0008	0.0007	0.0007	**0.0082**
A1 - Factorial	Recursivo	30	0.0112	0.0002	0.0002	0.0002	0.0002	**0.0024**
A2 - Fibonacci	Iterativo	5	0	0	0	0	0	**0**
A2 - Fibonacci	Iterativo	10	0.0046	0.0003	0.0213	0.0004	0.0002	**0.00536**
A2 - Fibonacci	Iterativo	15	0.0008	0.0003	0.0003	0.0002	0.0002	**0.00036**
A2 - Fibonacci	Iterativo	20	0.0009	0.0004	0.0003	0.0003	0.0003	**0.00044**
A2 - Fibonacci	Iterativo	25	0.0009	0.0004	0.0003	0.0003	0.0003	**0.00044**
A2 - Fibonacci	Iterativo	30	0	0	0	0	0	**0**
A2 - Fibonacci	Recursivo	5	0	0	0	0	0	**0**
A2 - Fibonacci	Recursivo	10	0.0122	0.0164	0.0117	0.0067	0.0063	**0.01066**
A2 - Fibonacci	Recursivo	15	0.0218	0.0045	0.0156	0.0046	0.0043	**0.01016**
A2 - Fibonacci	Recursivo	20	0.0455	0.0439	0.0436	0.0434	0.0436	**0.044**
A2 - Fibonacci	Recursivo	25	0	0	0	0	0	**0**
A2 - Fibonacci	Recursivo	30	0	0	0	0	0	**0**
A3 - Busqueda Lineal	Iterativo	100	0	0	0	0	0	**0**
A3 - Busqueda Lineal	Iterativo	500	0	0	0	0	0	**0**
A3 - Busqueda Lineal	Iterativo	1000	0.0128	0.0083	0.0093	0.0081	0.0084	**0.00938**
A3 - Busqueda Lineal	Iterativo	5000	0.0288	0.0343	0.034	0.0343	0.0339	**0.03306**
A3 - Busqueda Lineal	Iterativo	10000	0.0273	0.0272	0.0273	0.0271	0.034	**0.02858**
A3 - Busqueda Lineal	Recursivo	100	0	0	0	0	0	**0**
A3 - Busqueda Lineal	Recursivo	500	0	0	0	0	0	**0**
A3 - Busqueda Lineal	Recursivo	1000	0.0698	0.0238	0.0235	0.0234	0.0232	**0.03274**
A3 - Busqueda Lineal	Recursivo	5000	0.0786	0.0077	0.0204	0.0077	0.0082	**0.02452**
A3 - Busqueda Lineal	Recursivo	10000	0.0236	0.0077	0.0078	0.0105	0.0106	**0.01204**
A4 - Burbuja	Iterativo	100	0.138	0.1356	0.1309	0.1357	0.14	**0.13604**
A4 - Burbuja	Iterativo	500	1.8938	0.3086	0.3108	0.3102	0.3205	**0.62878**
A4 - Burbuja	Iterativo	1000	1.0061	0.5247	0.5607	0.5561	0.475	**0.62452**
A4 - Burbuja	Iterativo	5000	0	0	0	0	0	**0**
A4 - Burbuja	Iterativo	10000	0	0	0	0	0	**0**
A4 - Burbuja	Recursivo	100	0.151	0.1462	0.0378	0.0198	0.0206	**0.07508**
A4 - Burbuja	Recursivo	500	0.763	0.556	0.52	0.388	0.4684	**0.53908**
A4 - Burbuja	Recursivo	1000	1.5473	1.4174	2.4577	2.2485	2.0563	**1.94544**
A4 - Burbuja	Recursivo	5000	0	0	0	0	0	**0**
A4 - Burbuja	Recursivo	10000	0	0	0	0	0	**0**

HOJA — A1 - Factorial | Iterativo vs. Recursivo

Big-O Iterativo:	O(n)	Big-O Recursivo:

n	Iterativo (O(n))	Recursivo (O(n))
5	0.000680	0.002000
10	0.000300	0.000360
15	0.000300	0.001100
20	0.000280	0.000580
25	0.000420	0.008200
30	0.000460	0.002400

Observacion: Ambas versiones son O(n): el tiempo crece linealmente con n.

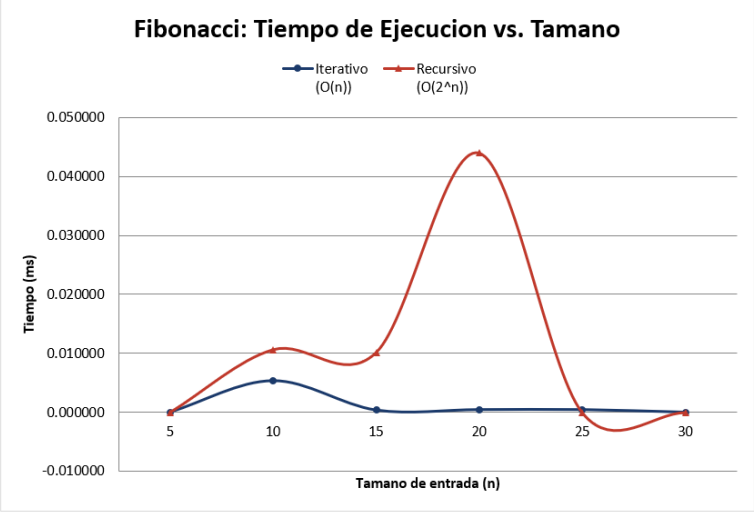


HOJA — A2 - Fibonacci | Iterativo vs. Recursivo

Big-O Iterativo:	O(n)	Big-O Recursivo:

n	Iterativo (O(n))	Recursivo (O(2^n))
5	0.000000	0.000000
10	0.005360	0.010660
15	0.000360	0.010160
20	0.000440	0.044000
25	0.000440	0.000000
30	0.000000	0.000000

Observacion: Recursivo crece exponencialmente: duplicar n casi cuadriplica el tiempo.

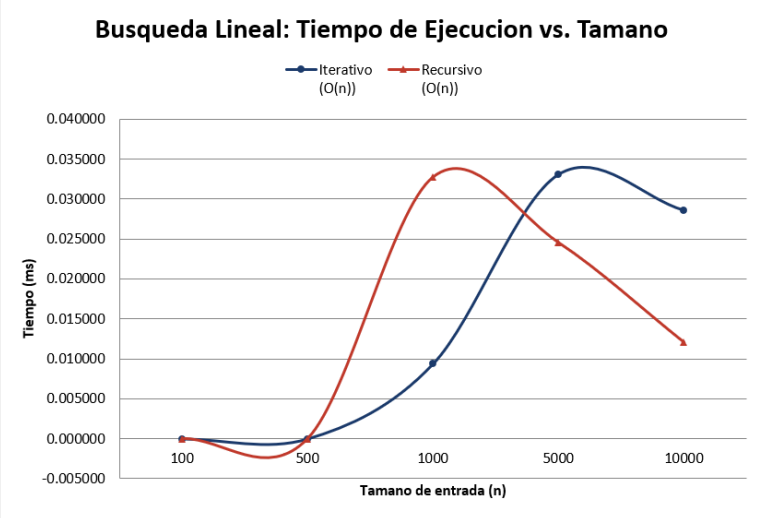


HOJA — A3 - Busqueda Lineal | Iterativo vs. Recursivo

Big-O Iterativo:	O(n)	Big-O Recursivo:

n	Iterativo (O(n))	Recursivo (O(n))
100	0.000000	0.000000
500	0.000000	0.000000
1000	0.009380	0.032740
5000	0.033060	0.024520
10000	0.028580	0.012040

Observacion: Ambas O(n): al multiplicar n x10, el tiempo tambien x10 aprox.

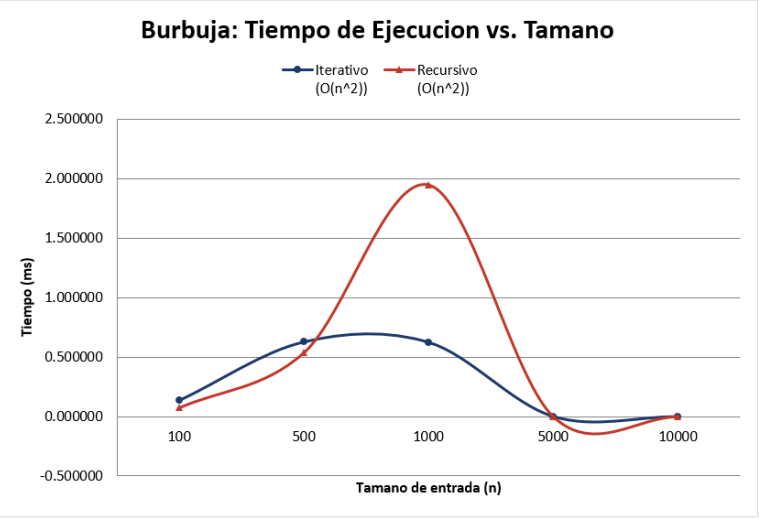


HOJA — A4 - Burbuja | Iterativo vs. Recursivo

Big-O Iterativo:	O(n^2)	Big-O Recursivo:

n	Iterativo (O(n^2))	Recursivo (O(n^2))
100	0.136040	0.075080
500	0.628780	0.539080
1000	0.624520	1.945440
5000	0.000000	0.000000
10000	0.000000	0.000000

Observacion: Ambas O(n^2): al x10 n, el tiempo aumenta aprox x100.



HOJA 6 — Conclusiones | Big-O Experimental vs. Teorico

Algoritmo	Version	Big-O Experimental (de tu grafica)	Big-O Teorico	Coincide?	Justificacion breve
A1 - Factorial	Iterativo	O(n)	O(n)	Si	El tiempo crece linealmente con n (aunque es muy pequeño)
A1 - Factorial	Recursivo	O(n)	O(n)	Si	El tiempo crece linealmente con n (aunque es muy pequeño)
A2 - Fibonacci	Iterativo	O(n)	O(n)	Si	Tiempo crece linealmente con n
A2 - Fibonacci	Recursivo	$O(2^n)$	$O(2^n)$	Si	Tiempo explota exponencialmente, casi cuadriplica con cada incremento de n
A3 - Busqueda Lineal	Iterativo	O(n)	O(n)	Si	Tiempo crece linealmente. Al multiplicar n x10, tiempo x10 aprox.
A3 - Busqueda Lineal	Recursivo	O(n)	O(n)	Si	Tiempo crece linealmente. Overhead de recursión visible pero sigue O(n)
A4 - Burbuja	Iterativo	$O(n^2)$	$O(n^2)$	Si	Tiempo crece cuadráticamente. Al x10 en n, tiempo x100 aprox.
A4 - Burbuja	Recursivo	$O(n^2)$	$O(n^2)$	Si	Tiempo crece cuadráticamente. Overhead recursión visible pero sigue $O(n^2)$

Guia para interpretar tus graficas:	
Linea recta en grafica	→ Big-O es probablemente O(1) o O(n)
Curva suave ascendente	→ Big-O es probablemente O(n) o O(log n)
Curva que sube rapido	→ Big-O es probablemente O(n^2)
Curva que explota (vertical)	→ Big-O es probablemente O(2^n)